

Omnisend Integration Playbook

Architecture, Pitfalls & AI Agent Guidelines • Hungry Hub Reference Guide

1. Executive Summary & Purpose

This document is a technical guide and post-mortem derived from the real-world integration of **Omnisend** into the **Hungry Hub** e-commerce platform (`hubhungry.vercel.app` — React, TypeScript, Supabase, Vercel Serverless).

Whenever an AI coding agent or engineer is tasked with integrating Omnisend into an e-commerce website, **read this document first**. Adhering to these patterns eliminates HTTP 409/422 conflicts, prevents broken Vercel serverless deployments, ensures accurate email revenue attribution, and guarantees a permanent **100% API sync health rate**.

2. Core Architecture & Implemented Pillars

Omnisend Integration Pillars				
1. Contacts Sync	2. Catalog Sync	3. Abandoned Cart Engine	4. Ecommerce Events	5. Sales Rev.
- Signup/Login - Phone (E.164) - Custom tags - Opt-in status	- Categories - Products - Daily 2am cron - Idempotent PUT	15m inactivity detection cron - Recovery links - Auto-dismiss	- Started Checkout - Placed Order - Add to Cart	- v3/orders - Email Attrib

Pillar 1: Contact Synchronization

Triggered on customer registration or profile updates. Formats phone numbers in strict **E.164** format (e.g., `+9779800000000`) and sets `channels.email.status: "subscribed"` with customer segmentation tags.

Pillar 2: Store Catalog Synchronization (Products & Categories)

Runs on a daily Vercel Cron at 02:00 UTC. Categories use a GET-first check to prevent duplicates. Products use idempotent `PUT /v3/products/{productId}` .

Pillar 3: Abandoned Cart Recovery Engine

Stores live carts in Supabase `abandoned_carts` . A cron job checks every 15 minutes for carts inactive >15 minutes and fires Omnisend's `added product to cart` event with a direct recovery link (`/recover-cart?cartId=...`). On checkout, the cart is marked `recovered` .

Pillar 4: Ecommerce Event Stream

Tracks user intent via `started checkout` (when dialog opens) and `placed order` .

Pillar 5: Store Sales & Automation Revenue Attribution

When a customer clicks an email and completes an order, submitting to `POST /v3/orders` enables Omnisend to attribute the revenue and display **"Sales: NPR"** directly in the automation workflow report.

3. Mistakes Made by Antigravity (Post-Mortem)

Mistake 1: Blind POST to /v3/categories Causing HTTP 409 Conflicts

What happened: The catalog sync tried to POST categories every day. Because categories like "Desserts" already existed, Omnisend returned `HTTP 409 Conflict: Category already exists`, dropping the dashboard health score from 100% to 87%.

The Fix: Always run `GET /v3/categories` first to collect existing IDs into a `Set`. Only POST categories that do not exist yet.

Mistake 2: Product POST vs Direct PUT

What happened: Calling POST on existing products threw 409 conflicts.

The Fix: Omnisend supports idempotent upsert via `PUT /v3/products/{productID}`. Always PUT first; only POST if PUT returns 404.

Mistake 3: Missing Required Schema Fields

- **Products:** Omnisend strictly requires `productUrl`, variant `id`, and variant `url`.
- **Orders (POST /v3/orders):** `orderNumber` MUST be an **integer (int64)**, NOT a string! `createdAt` is **mandatory** (ISO 8601 string). Currency fields (`orderSum`, `subTotalSum`) MUST be in **integer cents/paisa**.

Mistake 4: Vercel Subdirectory Routing Trap

What happened: Placing functions in `api/omnisend/*.ts` resulted in 404s on Vercel production.

The Fix: Maintain flat top-level bridge files (`api/omnisend-*.ts`) and configure explicit rewrites in `vercel.json`.

Mistake 5: Cross-Directory Imports in Vercel Serverless Functions

What happened: Importing helper functions across directories into `api/` broke Vercel's independent lambda bundler (`FUNCTION_INVOCATION_FAILED`).

The Fix: Inline all fetch routines and database calls directly within the serverless handler.

Mistake 6: Disconnected Checkout Trigger in Production

What happened: On Vercel, the React frontend places orders directly via Supabase RPC, skipping the Express backend. The frontend checkout dialog was never calling the Omnisend order endpoint, leaving Omnisend's automation Sales metric at `NPR 0.00`.

The Fix: Connect `fetch("/api/omnisend/placed-order")` fire-and-forget directly inside `checkout-dialog.tsx` on order success.

4. User Misconceptions & Dashboard Realities

The 3-Day Sliding Window Trap

Omnisend's API Log and Success Rate use a **rolling 72-hour (3-day) sliding window**. Old 409 error logs cannot be deleted manually. Once code is fixed to return 200 OK, you must let the 72-hour window slide forward. The errors naturally age out and the score climbs to 100%.

5. Strict Rules for Future AI Coding Agents

CRITICAL "DO NOT" RULES:

- **DO NOT** POST categories without checking `GET /v3/categories` first.
- **DO NOT** pass strings for `orderNumber` in `POST /v3/orders` (must be integer).
- **DO NOT** omit `createdAt` (ISO timestamp) in order payloads.
- **DO NOT** use cross-directory imports inside Vercel `api/` functions.
- **DO NOT** make Omnisend API calls blocking during checkout (always fire-and-forget).
- **DO NOT** panic when the health rate doesn't jump to 100% instantly (wait for the 3-day window).

MUST-DO CHECKLIST:

- Always use idempotent upserts (GET-check for categories, PUT for products).
- Always submit to both `POST /v3/orders` (Sales reporting) and `POST /api/events` (workflow triggers).
- Always format phone numbers to E.164 (e.g. `+977 ...`).
- Always multiply currency by 100 for `orderSum`, `subTotalSum`, and product prices in cents/paisa.

6. Canonical Working Code Snippets

A. Category Sync (Idempotent, Zero 409s)

```
const existingRes = await fetch("https://api.omnisend.com/v3/categories?limit=250", {
  headers: { "X-API-KEY": apiKey }
});
const existingData = await existingRes.json();
const existingIds = new Set((existingData.categories || []).map((c: any) => String(c.categoryID)));

for (const cat of categoriesToSync) {
  if (existingIds.has(cat.id)) continue; // Skip existing!
  await fetch("https://api.omnisend.com/v3/categories", {
    method: "POST",
    headers: { "X-API-KEY": apiKey, "Content-Type": "application/json" },
    body: JSON.stringify({ categoryID: cat.id, title: cat.title })
  });
}
```

B. Order & Revenue Attribution Dispatch

```
const orderSumCents = Math.round(totalPriceRupees * 100);
const parsedNum = parseInt(orderId.replace(/\D/g, ""), 10);
const orderNumber = !isNaN(parsedNum) && parsedNum > 0 ? parsedNum : Math.floor(Date.now() / 1000);

await fetch("https://api.omnisend.com/v3/orders", {
  method: "POST",
  headers: { "X-API-KEY": apiKey, "Content-Type": "application/json" },
  body: JSON.stringify({
    orderId: String(orderId),
    orderNumber: orderNumber,           // Must be int64!
    email: customerEmail,
    orderSum: orderSumCents,           // In cents/paisa!
    subTotalSum: orderSumCents,
    currency: "NPR",
    paymentStatus: "paid",
    fulfillmentStatus: "unfulfilled",
    createdAt: new Date().toISOString(), // REQUIRED!
    orderUrl: `${siteUrl}/orders`,
    products: lineItems.map((item) => ({
      productID: item.id || "1",
      variantID: item.id || "1",
      title: item.name,
      quantity: item.quantity,
      price: Math.round(item.price * 100) // In cents/paisa!
    }))
  })
});
```